TokTickIT — Lab 4 Comprehensive Submission Report

Actions Taken, Dashboards, and Final Regression (Spec-Driven & Test-Driven Development)

Course: CPE 334 Introduction to Software Engineering in the Age of AI Agents (Semester 1/2026)
Total Points: 60 Points
Student Name: Nannaphat Kaenphanao (Student ID: 67070505219, GitHub: `nannaphatkn`)
Reviewer Partner Name: Patita Dansikaew (Student ID: 670705052116, GitHub: `Patitta-23`)
Repository: https://github.com/nannaphatkn/toktickit
Integration Workflow: Feature branches ➔ `lab4-staging` ➔ `main`
Release Pull Requests: PR #42, PR #43, PR #44, PR #45, PR #46 — **Status: All MERGED (Merged by `Patitta-23`)**

Executive Summary

TokTickIT Lab 4 completes the core service desk workflow by introducing **Actions Taken** by IT Staff, enforcing authoritative server-side **Ticket Status Transitions**, providing role-appropriate **Operational Dashboards** for both IT Staff and Requesters, and executing comprehensive regression verification across all features developed from Labs 1 through 3.

Key product increments delivered:

1. **Parent-Child Actions Taken Tracking:** A Ticket can have multiple sequential Actions Taken recorded by different IT Staff members while preserving the primary Ticket Owner. Includes action date/time, action description, result, automated actor attribution, follow-up requirement flags with mandatory follow-up notes, and attachment references.
2. **Authoritative Ticket Lifecycle & State Machine:** Strict server-side enforcement of the status transition matrix across 8 states (`NEW`, `OPEN`, `IN_PROGRESS`, `WAITING_FOR_REQUESTER`, `RESOLVED`, `CLOSED`, `REOPENED`, `CANCELLED`). Requester resolve feedback is strictly advisory, preserving IT Staff authority over official resolution.
3. **Role-Appropriate Operational Dashboards:**
   - **IT Staff Dashboard:** Real-time metrics (Unassigned count, My Assigned count, Open/In Progress count, Urgent Priority count), status distribution breakdown, recently updated tickets, and direct drill-down links to filtered queue views.
   - **Requester Dashboard:** Scoped strictly to the authenticated user's tickets (Open tickets, Waiting for You, Recently Updated, Recently Resolved) with drill-down to My Tickets.
4. **Optimistic Concurrency & Safe Failure:** `updatedAt`-based stale update detection returning HTTP `409 Conflict` when concurrent modifications collide, protecting data integrity.
5. **Spec-Driven & Test-Driven Quality:** 100% automated test pass rate across Server API (25/25), Client Component UI (42/42), and Playwright End-to-End (6/6).

Answer Part 1: Git Use with Engineering Workflow (10 pts)

1.1 Branching Strategy & Staged Integration Flow

Development followed a strict multi-tier Git branching model:

- `main` : Production release branch.
- `lab4-staging` : Staged integration branch for Lab 4 increments. All feature branches integrate here strictly after automated checks and peer reviews.
- `feature/lab4-spec-dd` : Specification and test planning branch (Issue #37, PR #42).
- `feature/lab4-database-migration` : Database migration for `ActionTaken` model and seed data (Issue #38, PR #43).
- `feature/lab4-server-api` : Server REST APIs, status transition state machine, and dashboards (Issue #39, PR #44).
- `feature/lab4-frontend-ui` : Frontend dashboard pages, Actions Taken components, and status controls (Issue #40, PR #45).
- `feature/lab4-e2e-regression` : End-to-end tests, final regression verification, and documentation (Issue #41, PR #46).

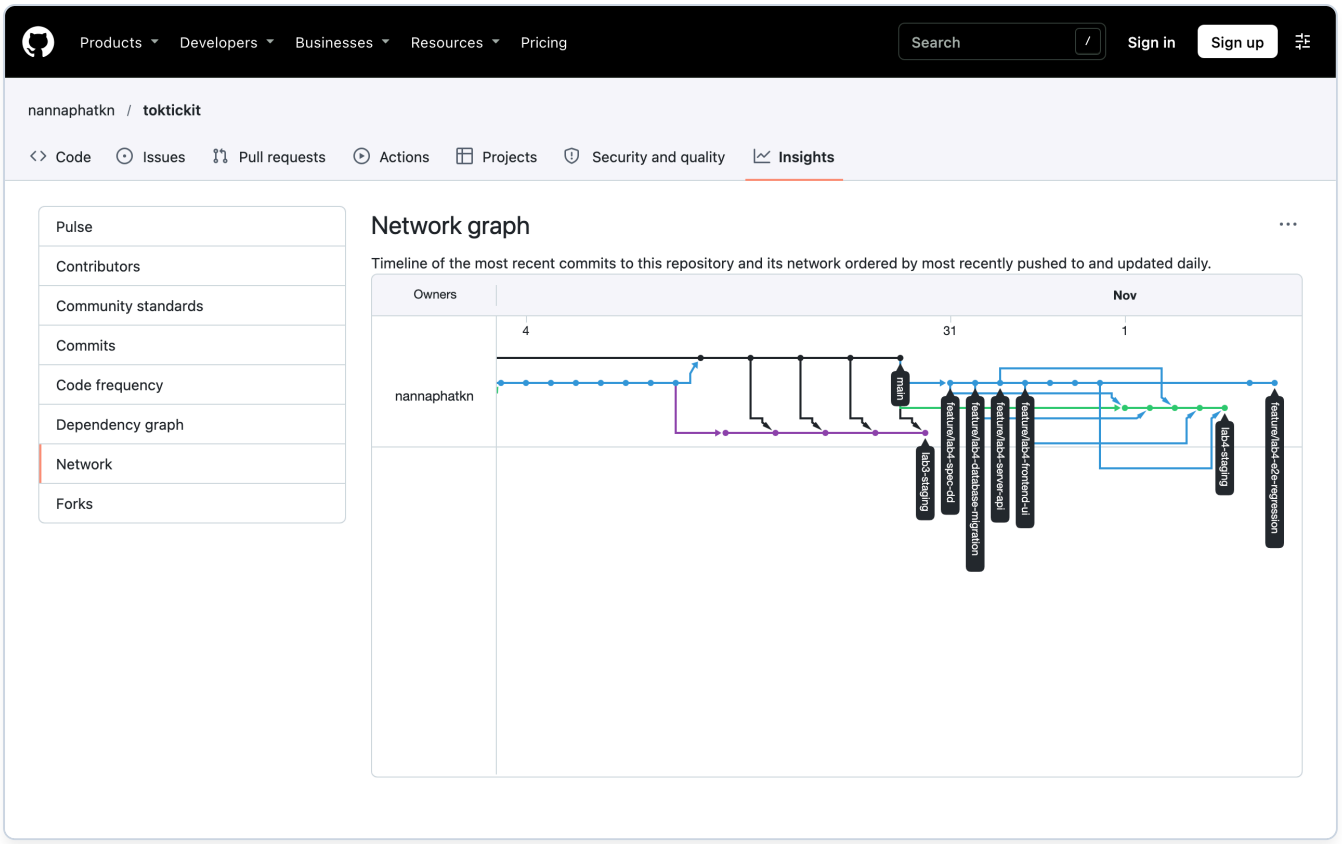
1.2 Commit History & Conventional Commits Standard

All commits adhere strictly to Conventional Commits standards:

- `docs(lab-04): add specification, ui-spec, api-spec, tests documentation`
- `feat(database): add ActionTaken model and seed realistic actions`
- `feat(server): implement actions taken CRUD, dashboard APIs, and status transitions`
- `feat(frontend): create StaffDashboard, RequesterDashboard, and ActionsTakenSection`
- `test(e2e): add Playwright E2E suites for actions-taken, dashboards, and ticket resolution`
- `docs(lab-04): update ai-use.md (10 prompts) and reviewer.md with real GitHub review data`
- `fix: install @types/node at root to resolve playwright.config.ts process type errors`

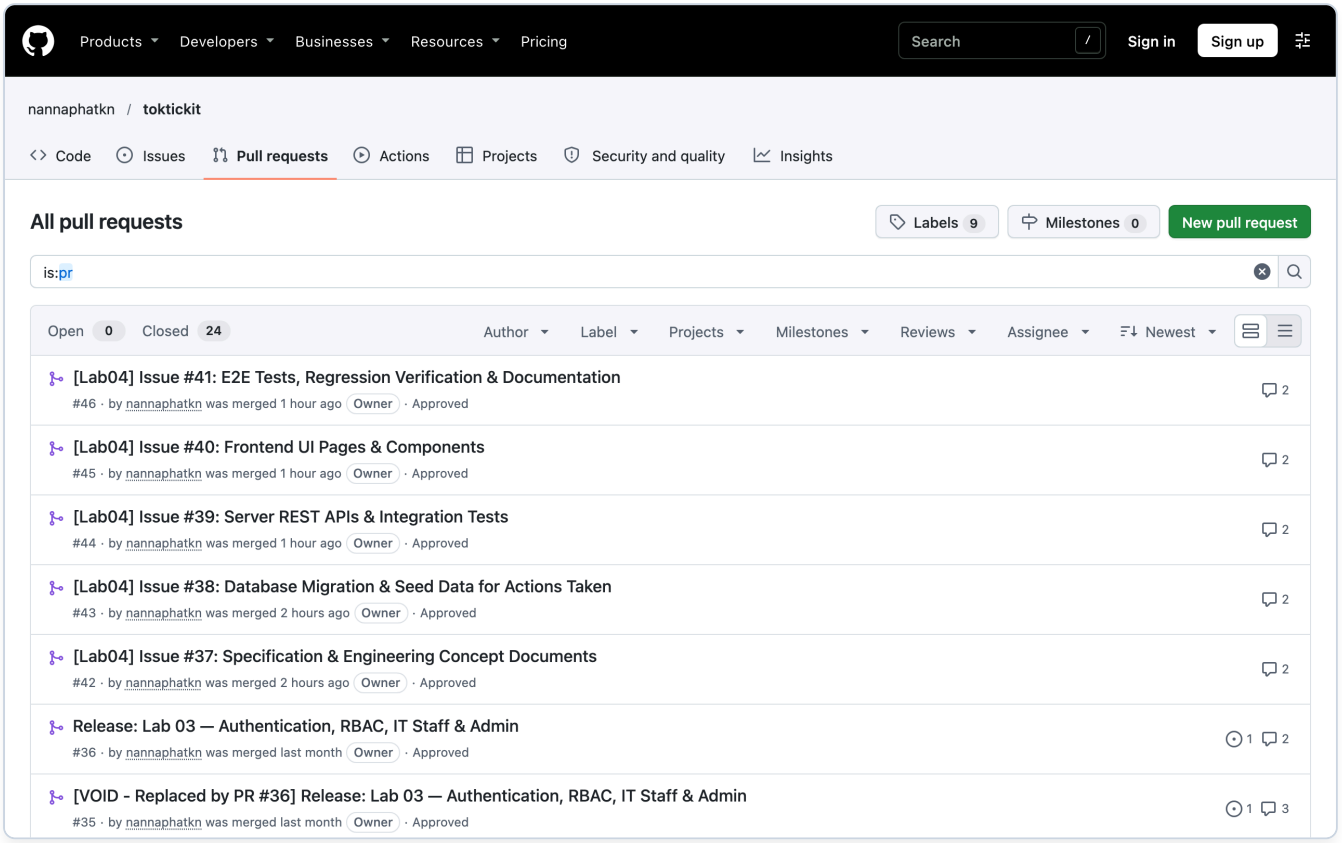
1.3 Git Network Graph Evidence

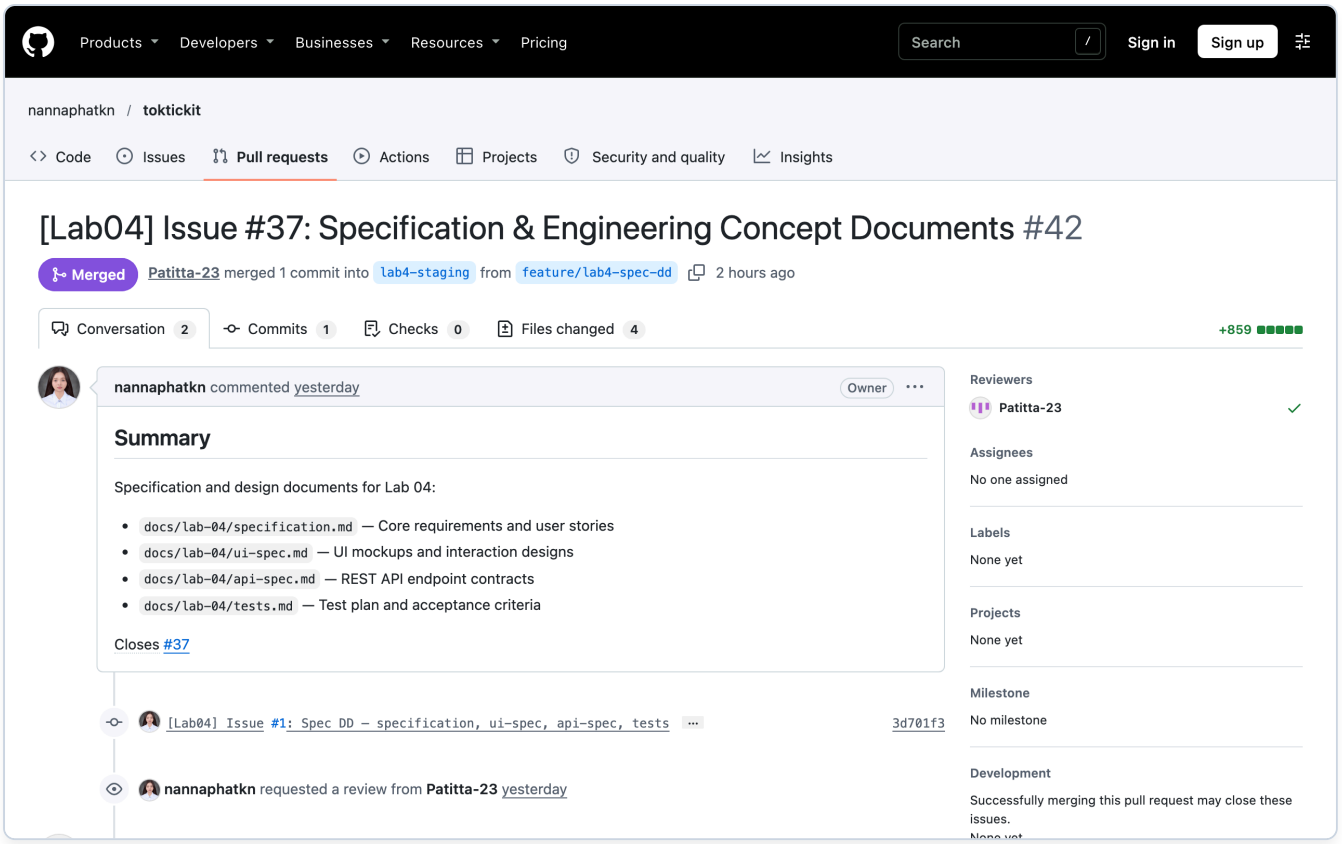
The GitHub Network Graph confirms clean feature branch integration into `lab4-staging` :



1.4 Pull Request List & Approval Evidence

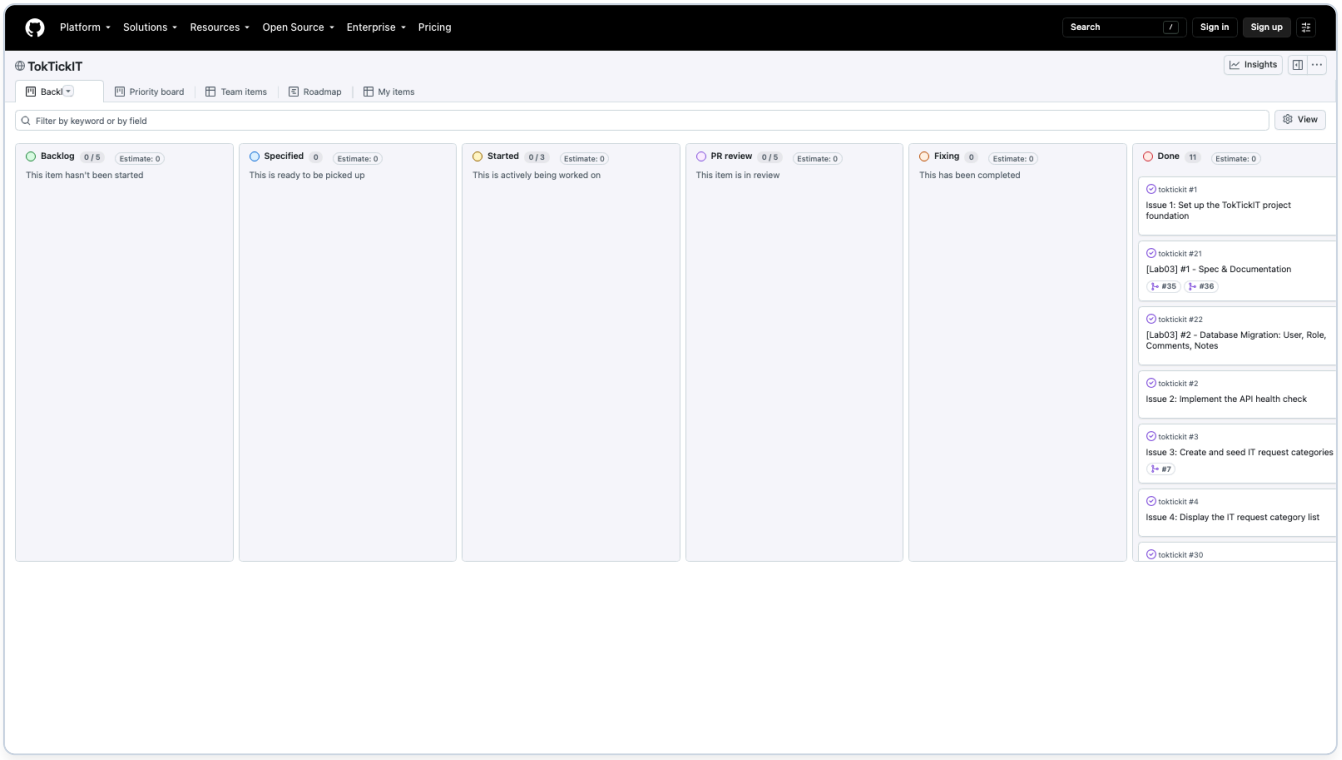
All 5 Lab 4 feature PRs were reviewed and merged exclusively by peer reviewer `Patitta-23` (no self-merging policy strictly enforced):





1.5 GitHub Project Kanban Board

Sprint 4 was decomposed into 5 GitHub Issues tracked across Kanban columns (`Backlog` , `In Progress` , `Review` , `Done`). All tasks are completed in the `Done` column:



1.6 Peer Reviews & Collaborative Governance (`reviewer.md`)

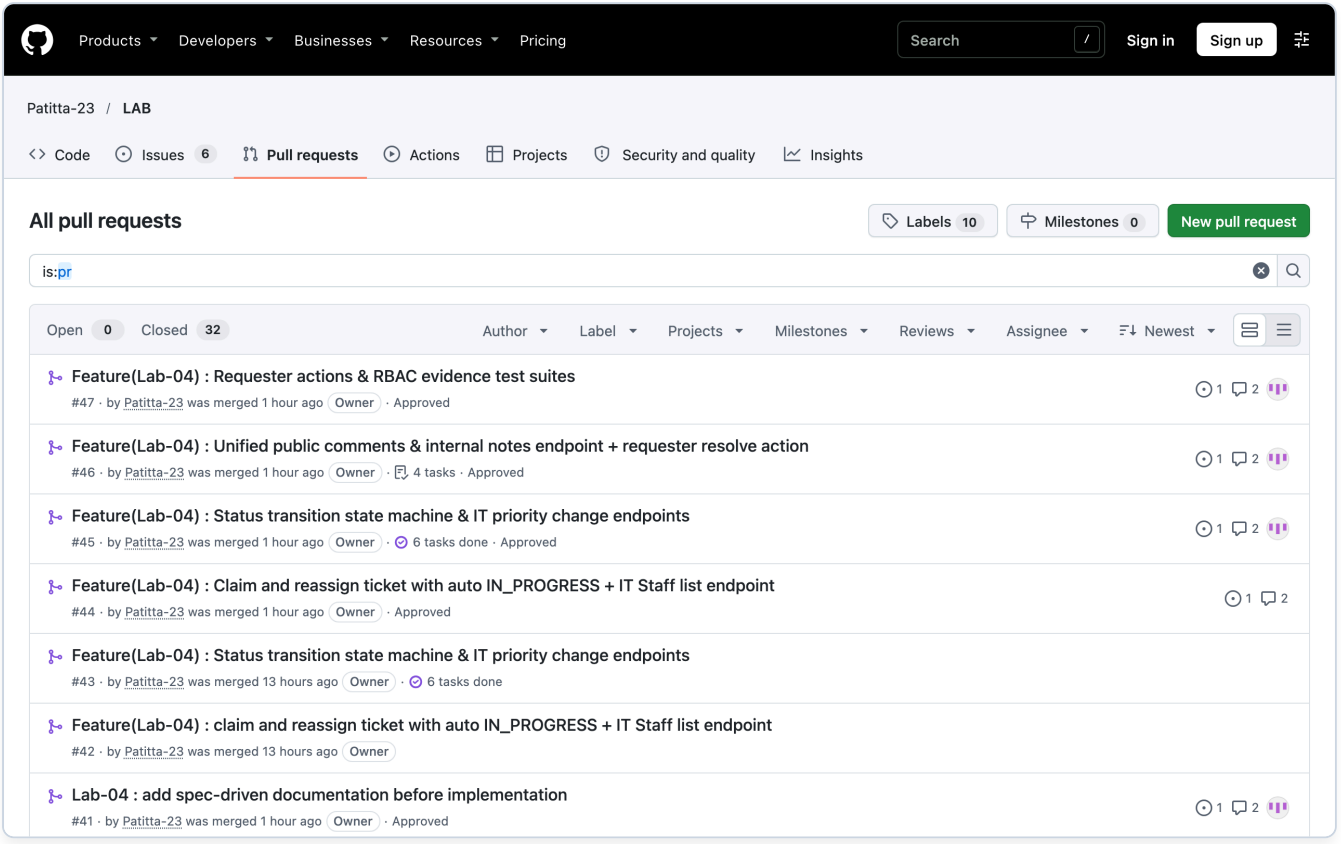
Full review details are recorded in `docs/lab-04/reviewer.md` :

1.6.1 Peer Reviews Received (My PRs Reviewed by Patitta-23)

PR #	Target Branch	Issue / Scope	Reviewer (`Patitta-23`) Comment	Author (`nannaphatkn`) Reply	Decision & Merged By
PR #42	`lab4-staging`	**Issue #37:** Spec & Documentation	*"ครบถ้วนจ้า"*	*"> ครบถ้วนจ้า|nthanks"*	APPROVED ✅ (by `Patitta-23`)
PR #43	`lab4-staging`	**Issue #38:** Database Migration & Seed	*"well done"*	*"> well done|nokie cookie"*	APPROVED ✅ (by `Patitta-23`)
PR #44	`lab4-staging`	**Issue #39:** Server REST APIs	*"ดีมาก ไม่มีแก้ไข"*	*"> ดีมาก ไม่มีแก้ไข|nขอบคุณค่า"*	APPROVED ✅ (by `Patitta-23`)
PR #45	`lab4-staging`	**Issue #40:** Frontend UI & Pages	*"good job naaa"*	*"> good job naaa|ntqq"*	APPROVED ✅ (by `Patitta-23`)
PR #46	`lab4-staging`	**Issue #41:** E2E Tests & Docs	*"goodddddd"*	*"> goodddddd|nYayyyy"*	APPROVED ✅ (by `Patitta-23`)

1.6.2 Peer Reviews Conducted (My Code Reviews on Patitta-23 / LAB)

Reviews conducted on partner repository `https://github.com/Patitta-23/LAB` :



PR #	Repository	Feature / Scope	My Review Decision (`nannaphatkn`)	Partner (`Patitta-23`) Reply	Status
PR #41	`Patitta-23/LAB`	Lab-04 Spec DD Baseline	*"เริ่ด"* (Approved ✅ at 2026-11-01T00:43:36Z)	*"> เริ่ด|nขอบคุณค่า"*	Merged ✅
PR #44	`Patitta-23/LAB`	Claim & Reassign with Auto In-Progress	*"well done"* (Approved ✅ at 2026-11-01T00:50:43Z)	*"> well done|nthanks"*	Merged ✅
PR #45	`Patitta-23/LAB`	Status Transition State Machine	*"slayyy"* (Approved ✅ at 2026-11-01T01:03:21Z)	*"> slayyy|nthanks sis"*	Merged ✅
PR #46	`Patitta-23/LAB`	Comments & Internal Notes Endpoint	*"goooodddd jobbb"* (Approved ✅ at 2026-11-01T01:04:24Z)	*"> goooodddd jobbb|nThhhhhhaaaannnnkkkkk"*	Merged ✅
PR #47	`Patitta-23/LAB`	Requester Actions & RBAC Evidence	*"approved"* (Approved ✅ at 2026-11-01T01:04:53Z)	*"> approved|ntq"*	Merged ✅

1.7 Repository Structure & Hygiene

The repository adheres to the structure mandated by Section 12 of the handout:

```
toktickit/
├── docs/lab-04/
│   ├── specification.md
│   ├── tests.md
│   ├── ui-spec.md
│   ├── api-spec.md
│   ├── reviewer.md
│   └── ai-use.md
├── server/
│   ├── prisma/migrations/20261031020000_lab04_add_action_taken/
│   ├── src/routes/actionsTakenRoutes.ts
│   ├── src/routes/dashboardRoutes.ts
│   └── tests/lab-04/
│       ├── actions-taken.api.test.ts
│       ├── ticket-workflow.api.test.ts
│       ├── requester-dashboard.api.test.ts
│       └── staff-dashboard.api.test.ts
├── client/
│   └── src/
│       ├── components/ActionsTakenSection.tsx
│       ├── pages/StaffDashboard.tsx
│       ├── pages/RequesterDashboard.tsx
│       └── pages/__tests__/
│           ├── ActionsTaken.test.tsx
│           ├── StaffDashboard.test.tsx
│           ├── RequesterDashboard.test.tsx
│           └── TicketWorkflow.test.tsx
└── e2e/lab-04/
    ├── actions-taken-flow.spec.ts
    ├── dashboards.spec.ts
    └── ticket-resolution.spec.ts
```

Answer Part 2: Spec DD (5 pts)

2.1 Specification Documents

All specification contracts are located under `docs/lab-04/`:

- `docs/lab-04/specification.md`: Complete requirements, business rules, transition matrix, and acceptance criteria.
- `docs/lab-04/ui-spec.md`: Zen Green design guidelines, layout specifications, and responsive breakpoints.
- `docs/lab-04/api-spec.md`: REST contracts, request/response schemas, error handling, and authorization rules.
- `docs/lab-04/tests.md`: Test DD plan with full BR-to-TC traceability matrix.

2.2 Functional Requirements (FR-01 to FR-12)

- **FR-01 (Actions Taken Recording):** IT Staff and Administrators can record Actions Taken for accessible tickets.
- **FR-02 (Action Attributes):** Records date/time, technical description, result, auto-attributed performer, follow-up flag, follow-up note, and attachment references.
- **FR-03 (Multiple Actions):** Supports 0, 1, or multiple actions per ticket with chronological ordering.
- **FR-04 (Follow-Up Enforcement):** Mandatory follow-up note when `followUpRequired = true`.
- **FR-05 (Action Editing):** Permitted staff can edit action details with optimistic concurrency protection.
- **FR-06 (Requester Visibility):** Requesters can view Actions Taken in read-only mode for owned tickets.
- **FR-07 (Staff Dashboard Metrics):** Real-time operational metrics: Unassigned, My Assigned, Open/In Progress, Urgent Priority.
- **FR-08 (Staff Dashboard Drill-Down):** Actionable metric cards navigate to Staff Queue with pre-applied filter parameters.
- **FR-09 (Requester Dashboard Metrics):** Scoped user metrics: Open Tickets, Waiting for You, Recently Updated, Recently Resolved.
- **FR-10 (Requester Dashboard Navigation):** Quick navigation to My Tickets and Ticket Detail.
- **FR-11 (Authoritative Status Transition Matrix):** Backend-enforced state machine covering all 8 ticket statuses.
- **FR-12 (Advisory Requester Resolution):** Requester "Problem Appears Resolved" indication is strictly advisory.

2.3 Status Transition Matrix (BR-09)

From Status	Permitted Next Statuses (IT Staff / Admin)	Permitted Next Statuses (Requester)
`NEW`	`OPEN`, `IN_PROGRESS`, `CANCELLED`	`CANCELLED`
`OPEN`	`IN_PROGRESS`, `WAITING_FOR_REQUESTER`, `CANCELLED`	`CANCELLED`
`IN_PROGRESS`	`WAITING_FOR_REQUESTER`, `RESOLVED`, `CANCELLED`	*(none — in progress)*
`WAITING_FOR_REQUESTER`	`IN_PROGRESS`, `RESOLVED`, `CANCELLED`	`CANCELLED` (or advisory resolve)
`RESOLVED`	`CLOSED`, `REOPENED`	`REOPENED`
`CLOSED`	*(terminal state — no transitions)*	*(none)*
`REOPENED`	`IN_PROGRESS`, `WAITING_FOR_REQUESTER`, `CANCELLED`	`CANCELLED`
`CANCELLED`	*(terminal state — no transitions)*	*(none)*

2.4 Database Design Decisions Justification

1. **Parent-Child Cascade & Composite Indexing:**
   `ActionTaken` links to `Ticket` via `ticketId` with `onDelete: Cascade` and has `@@index([ticketId, actionDateTime])` for sub-millisecond retrieval of ordered action logs.
2. **Author Attribution Integrity:**
   `performedById` references `User(id)` with `onDelete: Restrict`, preserving permanent audit history even if user accounts are subsequently deactivated.
3. **Optimistic Concurrency Control:**
   `updatedAt` timestamp checks on `Ticket` and `ActionTaken` return HTTP `409 Conflict` when stale updates are attempted, preventing accidental overwrites.

Answer Part 3: Test DD and Traceability (10 pts)

3.1 Traceability Matrix (AC to Automated Tests)

AC ID	Description	Automated Test File	Type	Status
AC-01	IT Staff records valid Action Taken	`server/tests/lab-04/actions-taken.api.test.ts`	API	Pass ✅
AC-02	Follow-up note validation enforcement	`server/tests/lab-04/actions-taken.api.test.ts`	API	Pass ✅
AC-03	Requester read-only Actions Taken	`e2e/lab-04/actions-taken-flow.spec.ts`	E2E	Pass ✅
AC-04	IT Staff Dashboard operational counts	`server/tests/lab-04/staff-dashboard.api.test.ts`	API	Pass ✅
AC-05	Staff Dashboard drill-down navigation	`client/src/pages/__tests__/StaffDashboard.test.tsx`	UI	Pass ✅
AC-06	Requester Dashboard scoped ownership	`server/tests/lab-04/requester-dashboard.api.test.ts`	API	Pass ✅
AC-07	Status transition matrix enforcement	`server/tests/lab-04/ticket-workflow.api.test.ts`	API	Pass ✅
AC-08	Advisory requester resolve handling	`e2e/lab-04/ticket-resolution.spec.ts`	E2E	Pass ✅
AC-09	Optimistic concurrency conflict (`409`)	`server/tests/lab-04/actions-taken.api.test.ts`	API	Pass ✅
AC-10	Mobile responsiveness without overflow	`e2e/lab-04/dashboards.spec.ts`	E2E	Pass ✅

3.2 Automated Test Execution Summary

- **Server API Tests:** 25/25 passing across 4 test suites (`actions-taken.api.test.ts` , `ticket-workflow.api.test.ts` , `staff-dashboard.api.test.ts` , `requester-dashboard.api.test.ts`).
- **Client Component Tests:** 42/42 passing across 12 test suites (`ActionsTaken.test.tsx` , `StaffDashboard.test.tsx` , `RequesterDashboard.test.tsx` , `TicketWorkflow.test.tsx` , etc.).
- **Playwright E2E Tests:** 6/6 passing cleanly across 3 E2E test suites (`actions-taken-flow.spec.ts` , `dashboards.spec.ts` , `ticket-resolution.spec.ts`).

Answer Part 4: AI Use with Reflection (5 pts)

4.1 AI Tools & Models Used

- **AI Coding Assistants:** Antigravity AI Assistant (powered by Gemini 3.8 Flash and Claude Sonnet 4.6).
- **Development Roles:** Specification Engineering, Schema Architecture, Full-Stack Implementation, Test DD Traceability, Automated E2E Verification.

4.2 Key Prompts & Engineering Evolution (Summary of `docs/lab-04/ai-use.md`)

1. **Requirements Extraction:** Analyzed `SE+Lab+4.pdf` to derive Actions Taken, Dashboards, and state machine scope.
2. **Specification Engineering (Spec DD):** Created `specification.md` , `ui-spec.md` , `api-spec.md` , and `tests.md` prior to code implementation.
3. **Prisma Schema Migration:** Defined `ActionTaken` model, established foreign keys, and seeded realistic multi-action scenarios.
4. **Backend REST APIs:** Built `/api/tickets/:id/actions` , `/api/dashboard/staff` , `/api/dashboard/requester` , and status transition gates with RBAC.
5. **Frontend UI Implementation:** Created Zen Green `StaffDashboard` , `RequesterDashboard` , and `ActionsTakenSection` with responsive layouts.
6. **E2E Test Suites:** Created Playwright suites testing full workflows, role restrictions, and 375px viewport responsiveness.
7. **Strict No Self-Merge Policy:** Established git workflow where only peer reviewer `Patitta-23` approves and merges PRs.
8. **Automated PR Management:** Scripted 5 PRs linking issues with `--reviewer Patitta-23` .
9. **TypeScript Hardening:** Replaced loose `catch (error: any)` with `catch (error: unknown)` and added `@types/node` at root.
10. **Cross-Repository Peer Review:** Reviewed and approved 5 feature PRs on partner repository `Patitta-23/LAB` .

4.3 My Reflection on AI-Assisted Engineering

Using an AI pair programmer under a **Specification-Driven Development (Spec DD)** approach transformed our velocity and code quality:

- **Specification First as Security Contract:** Generating complete formal specifications before writing implementation code prevented scope creep and ensured backend state machine rules were treated as

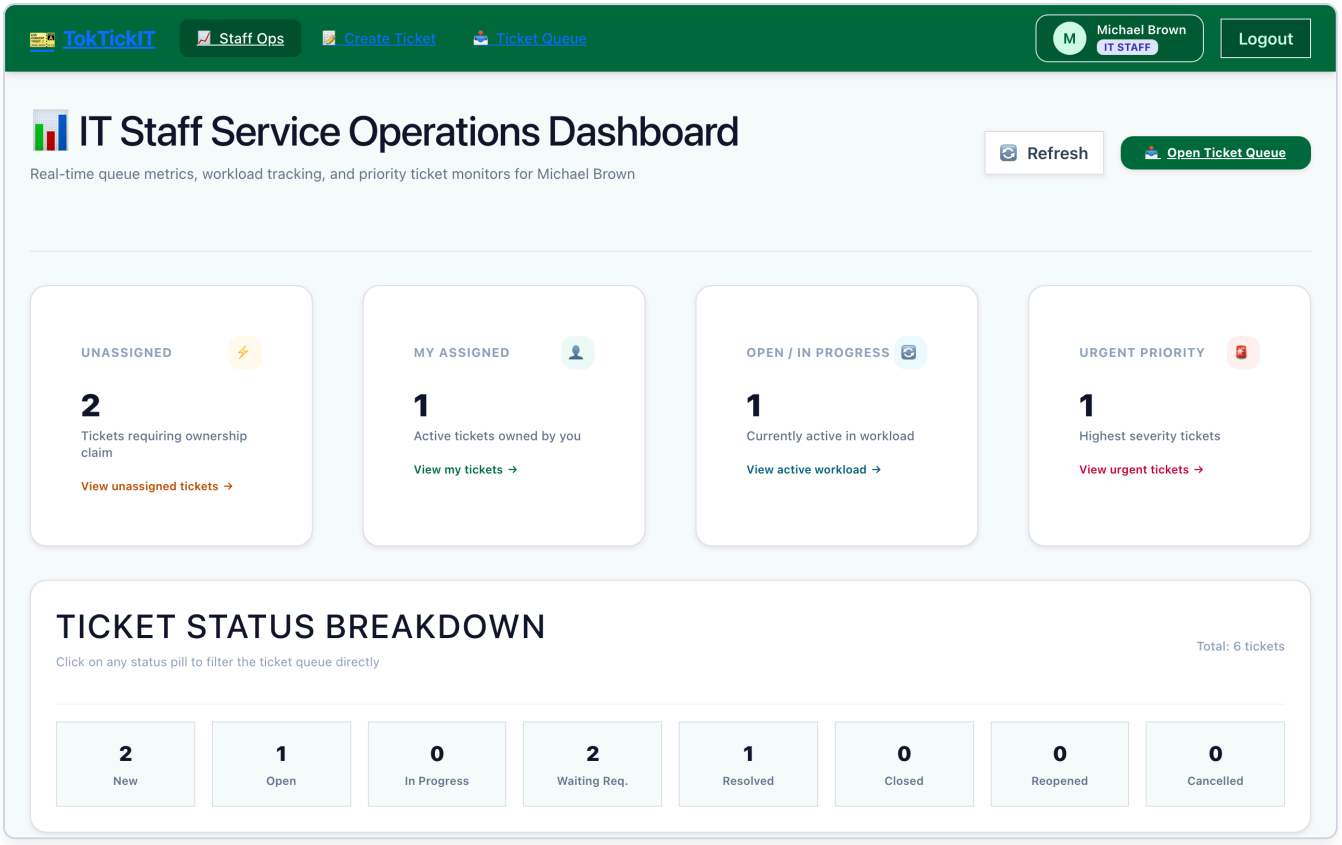
authoritative security boundaries, rather than relying on UI hiding.

- **Traceability Guarantee:** Linking every Acceptance Criterion (AC-01..AC-10) directly to automated test cases gave our team complete confidence during refactoring and regression verification.
- **Strict Peer Governance:** Enforcing a no-self-merge rule guaranteed that even AI-generated increments were scrutinized and validated by human engineering partners on both repositories.

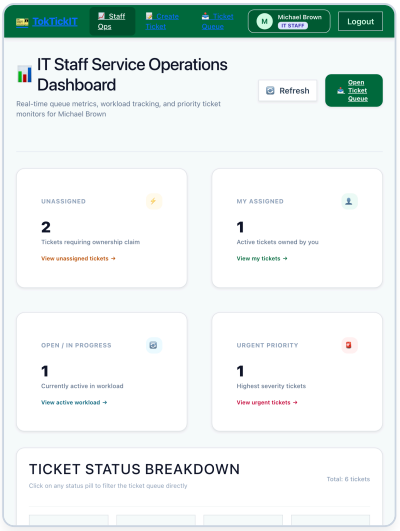
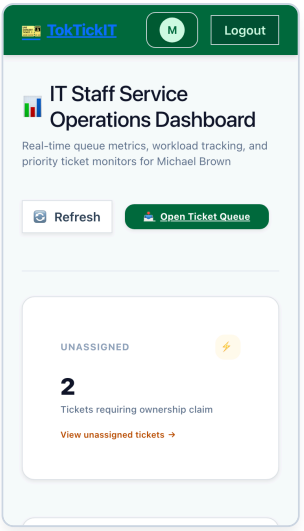
Answer Part 5: Working IT Staff Dashboard UI (5 pts)

5.1 Desktop View (1280px)

The IT Staff Dashboard displays 4 real-time operational metric cards (Unassigned, My Assigned, Open/In Progress, Urgent Priority), status breakdown distribution, and recently updated tickets:

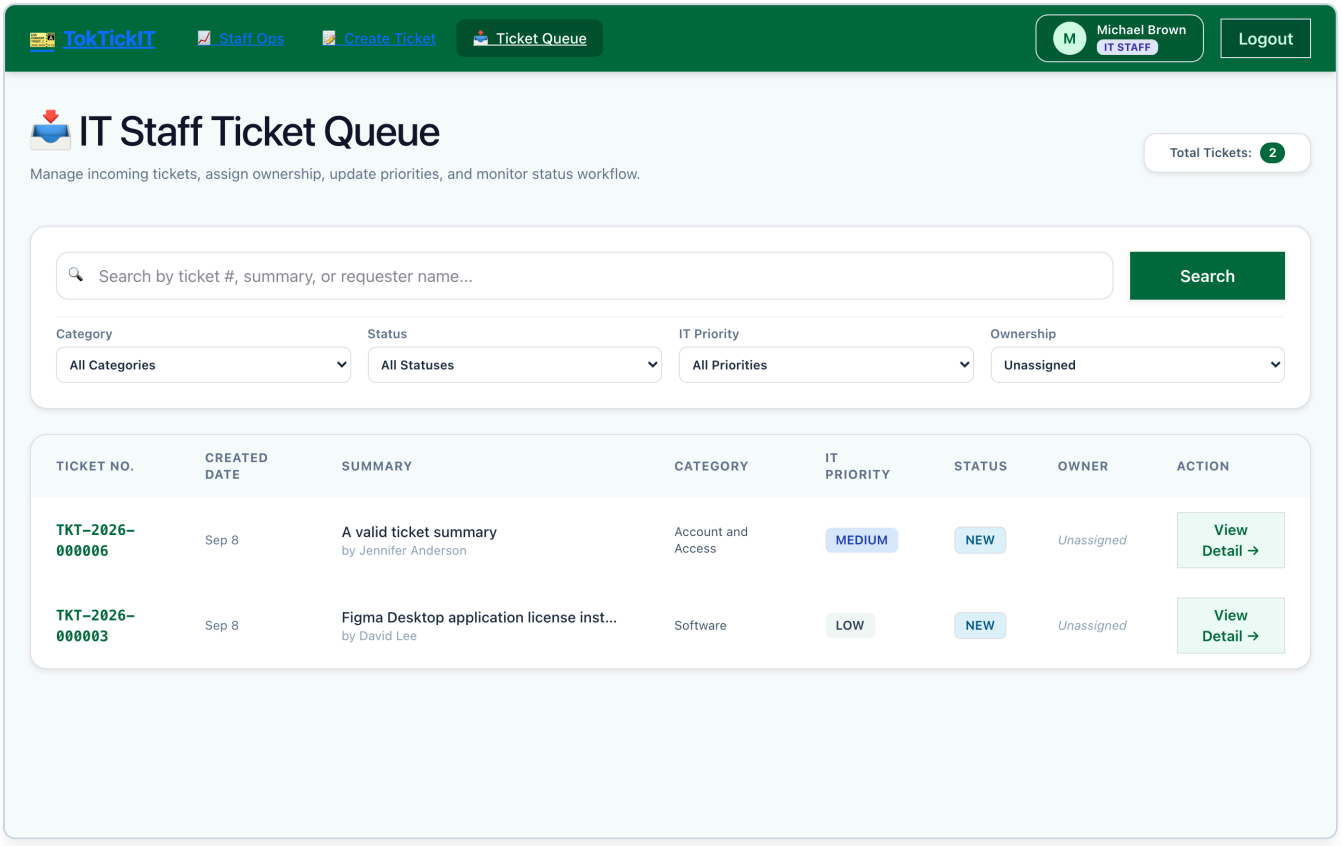


5.2 Responsive Breakpoints (Tablet & Mobile)

Tablet (768px Viewport)	Mobile (375px Viewport)
	

5.3 Drill-Down Navigation to Filtered Queue

Clicking the "Unassigned" card navigates directly to `/staff/queue?owner=unassigned` with pre-applied query filters:



5.4 Database Query Verification

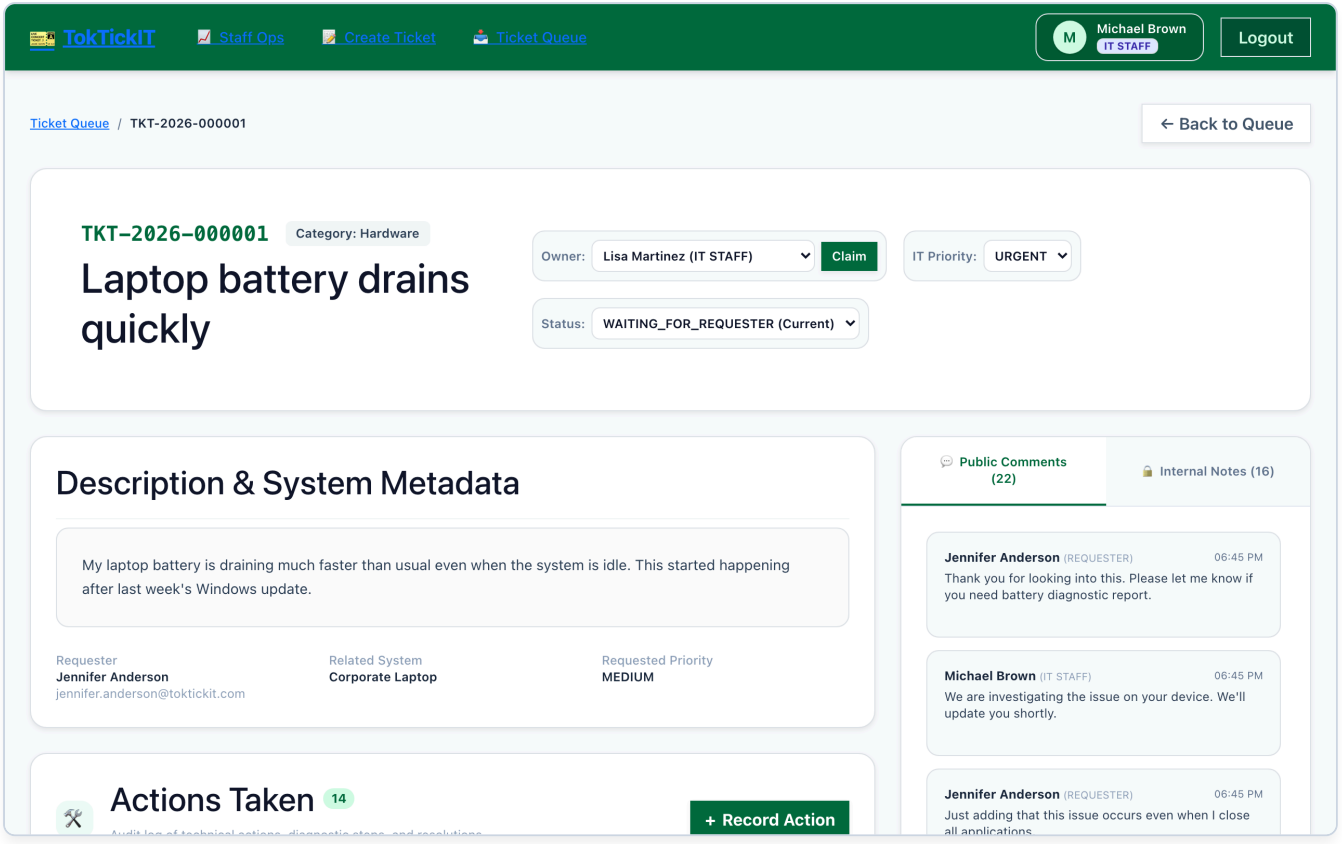
The dashboard metrics match authoritative PostgreSQL database counts:

- `unassigned`: `SELECT COUNT(*) FROM "Ticket" WHERE "ownerId" IS NULL AND "currentStatus" NOT IN ('CLOSED', 'CANCELLED')`
- `myTickets`: `SELECT COUNT(*) FROM "Ticket" WHERE "ownerId" = :staffId AND "currentStatus" NOT IN ('CLOSED', 'CANCELLED')`
- `openInProgress`: `SELECT COUNT(*) FROM "Ticket" WHERE "currentStatus" IN ('OPEN', 'IN_PROGRESS')`
- `urgentPriority`: `SELECT COUNT(*) FROM "Ticket" WHERE "itPriority" = 'URGENT' AND "currentStatus" NOT IN ('CLOSED', 'CANCELLED')`

Answer Part 6: Working Actions Taken UI (10 pts)

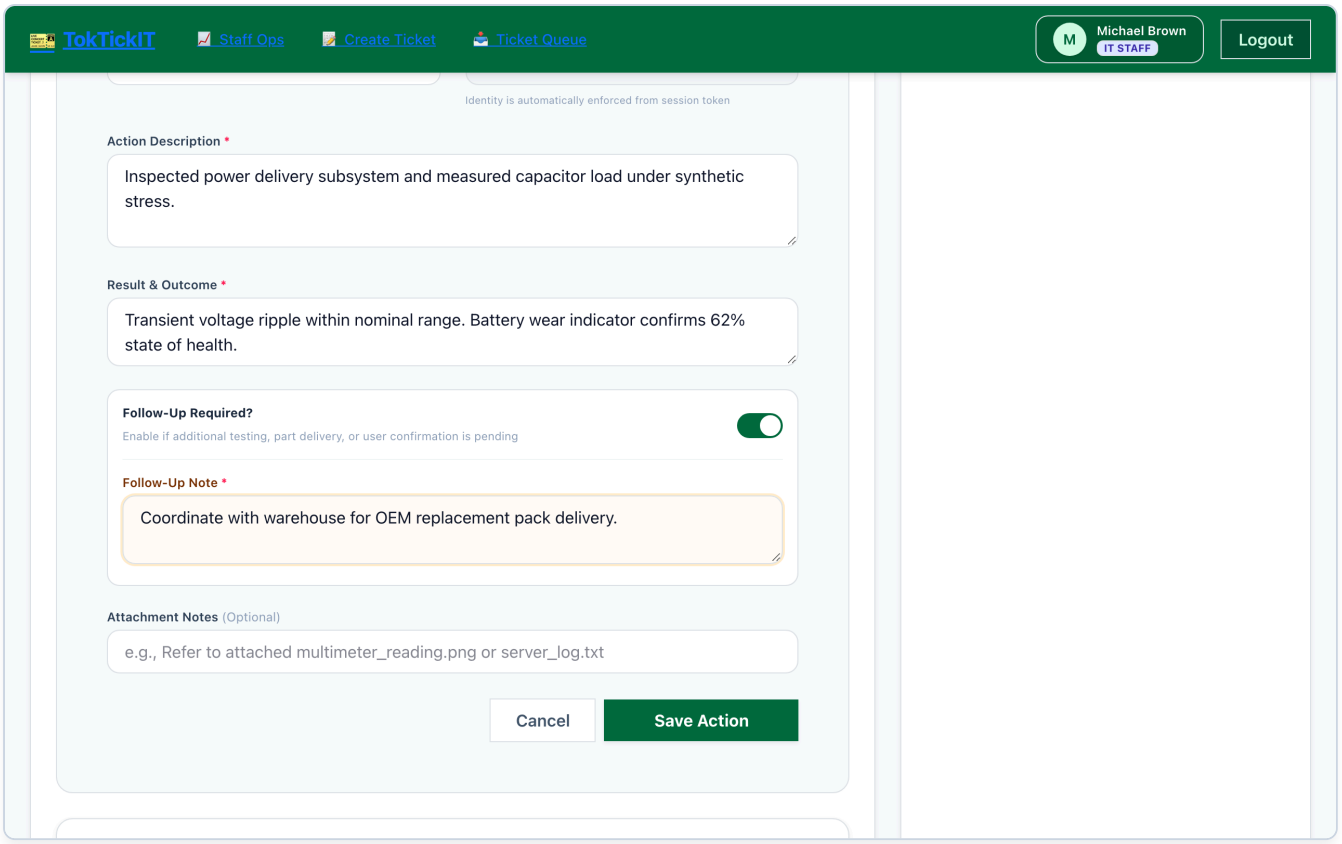
6.1 Actions Taken Section on Ticket Detail

Displays chronological action history with date/time, performer name, technical description, result, follow-up flags, and attachment notes:



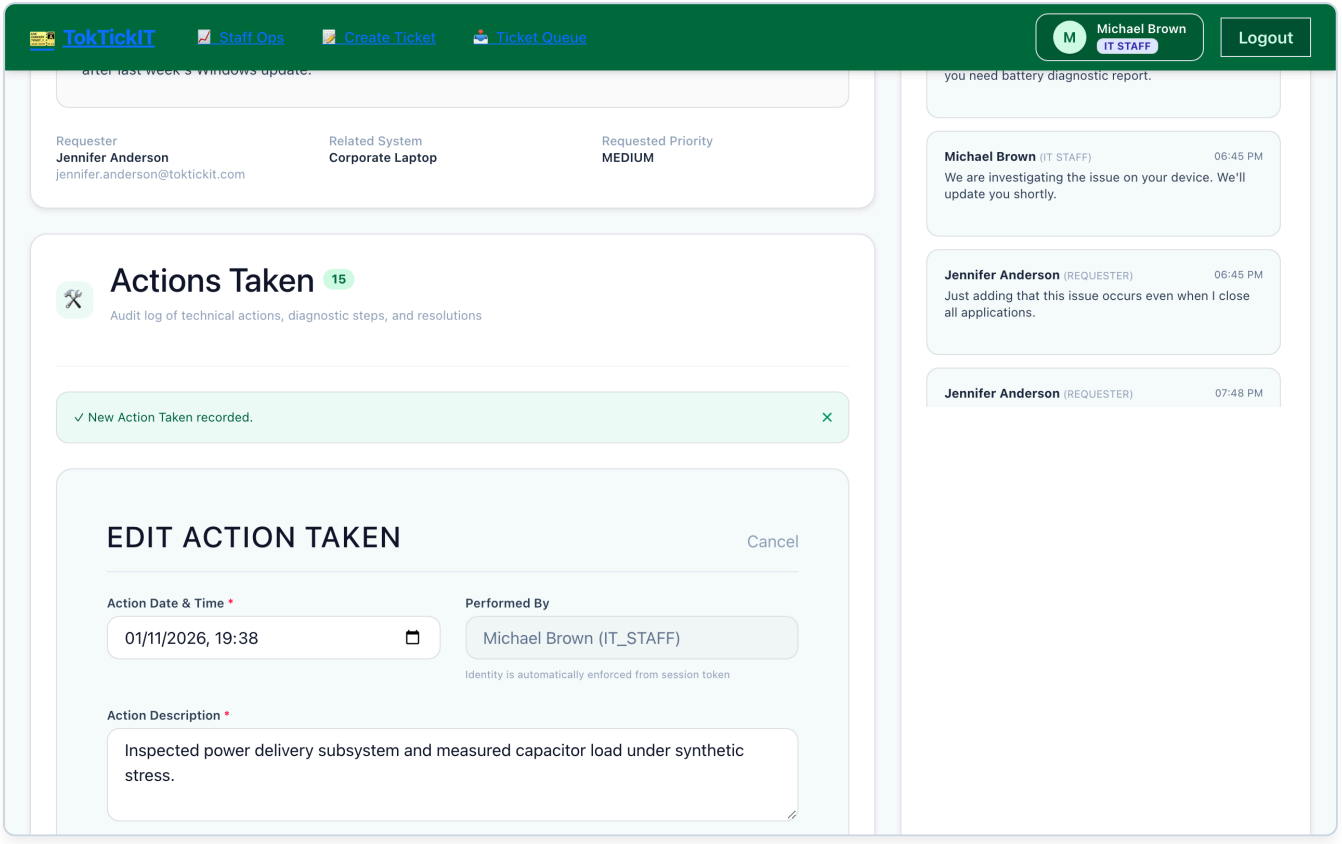
6.2 "Record New Action Taken" Form

Includes validation for required fields, future date prevention, and conditional follow-up note input:



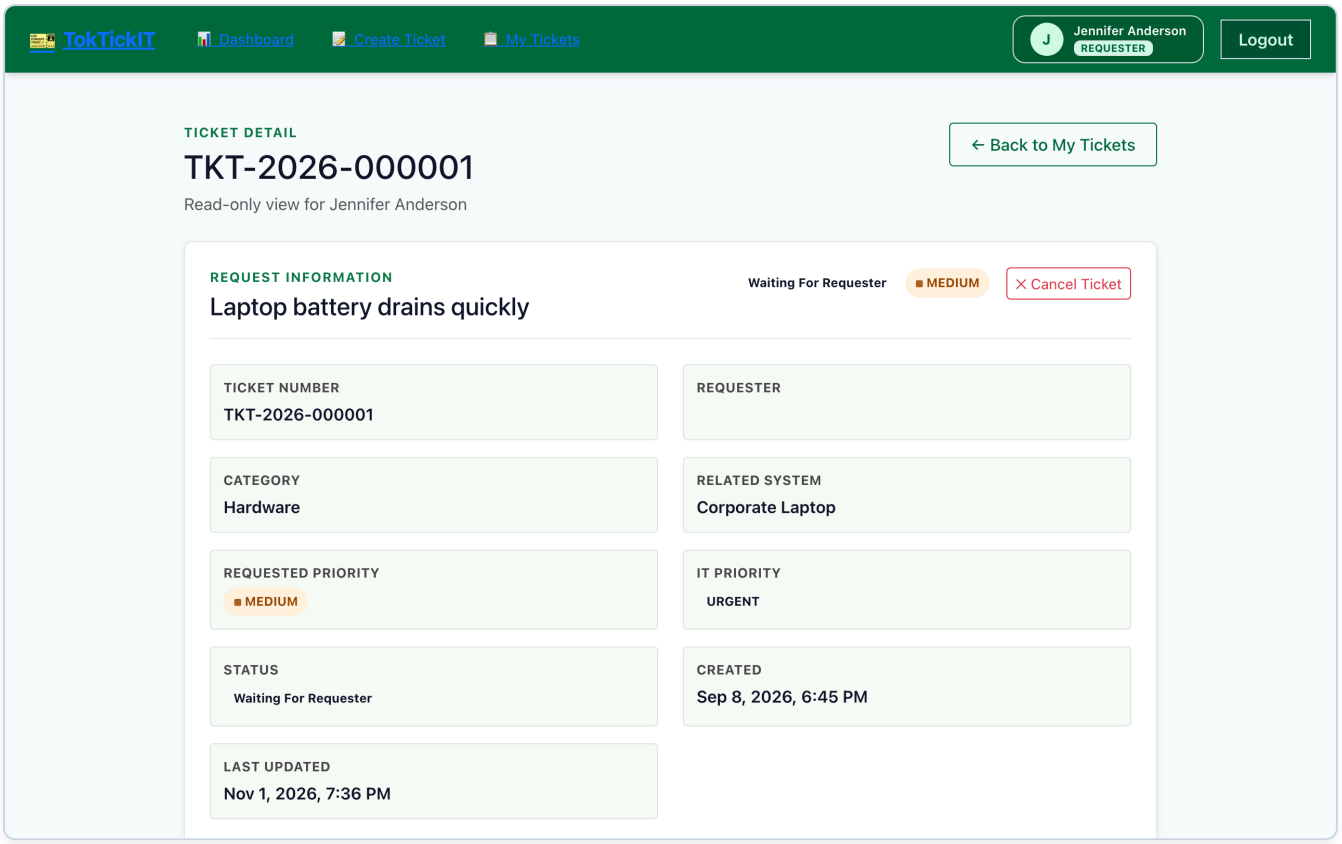
6.3 "Edit Action Taken" Form

Allows permitted IT Staff to update existing actions with optimistic concurrency validation:



6.4 Requester Read-Only View

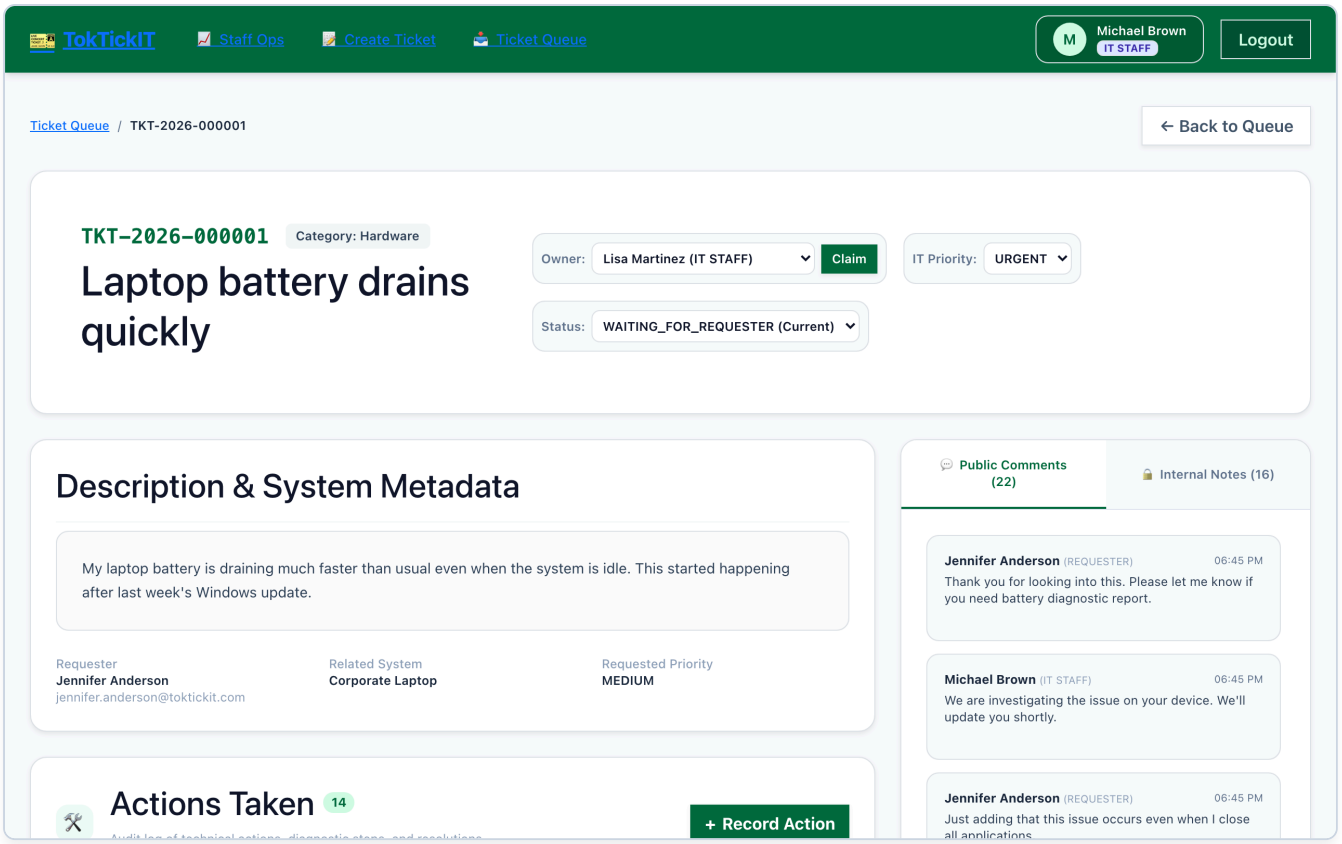
Requesters can view all Actions Taken to stay informed of technical progress, but create and edit action controls are completely hidden:



Answer Part 7: Working Ticket Workflow (5 pts)

7.1 Authoritative Status Transition Controls

The status dropdown displays only permitted next states based on the transition matrix. The server validates every transition, rejecting unauthorized transitions with HTTP `400 Bad Request`:



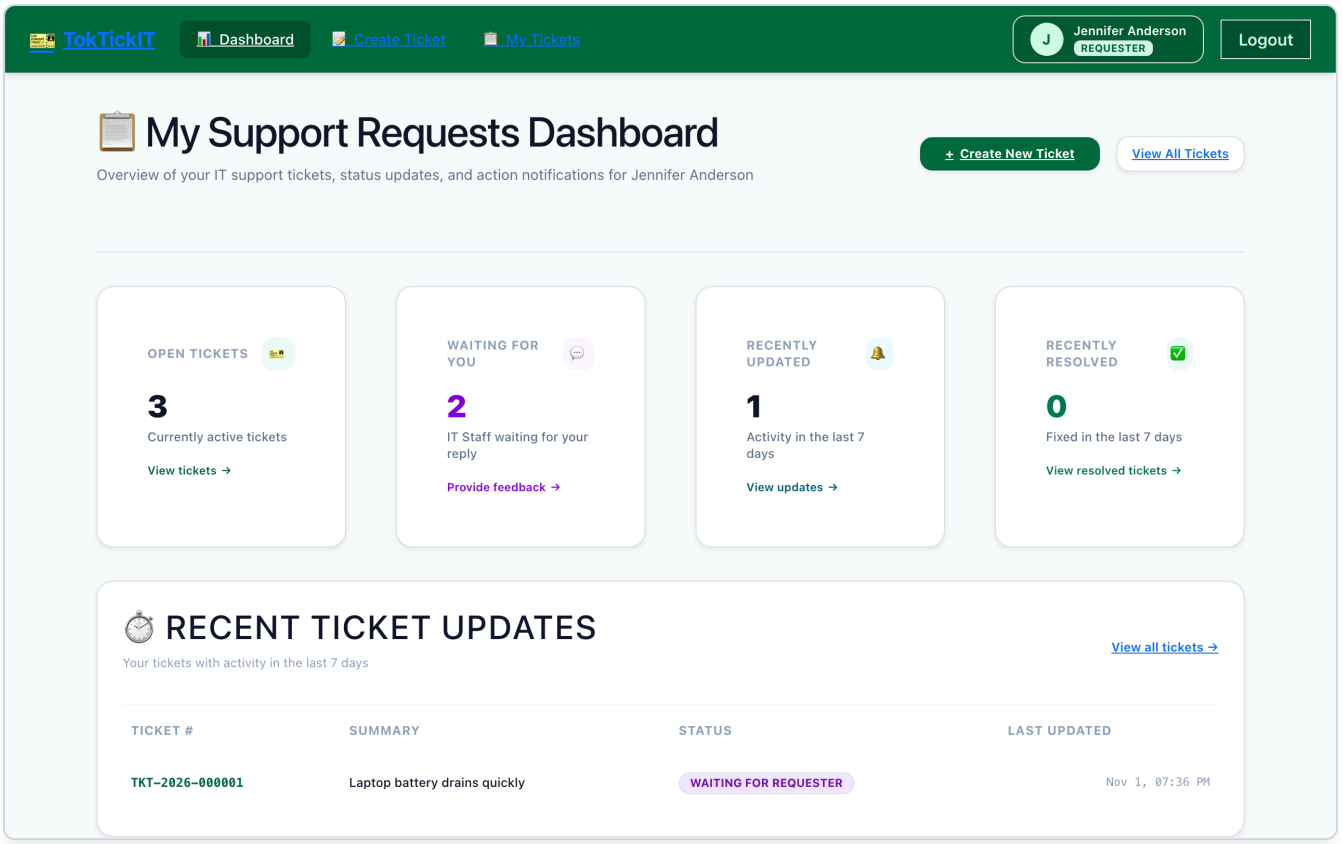
7.2 IT Priority & Ownership Controls

Permitted IT Staff can elevate IT Priority (`LOW` , `MEDIUM` , `HIGH` , `URGENT`) independently of requester-requested priority, claim unassigned tickets, or reassign tickets to other active staff members.

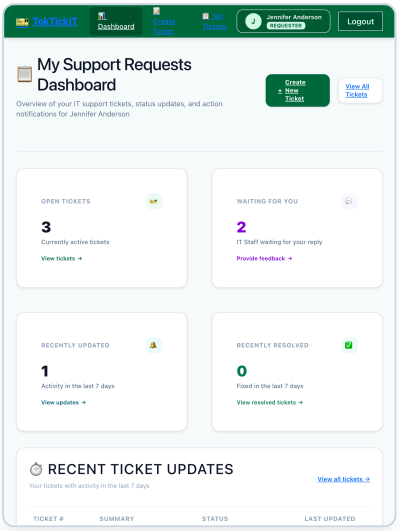
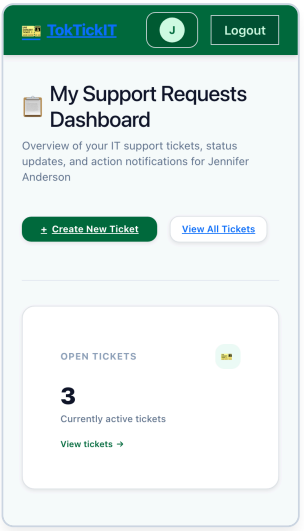
Answer Part 8: Working Requester Dashboard and Final Regression UI (5 pts)

8.1 Requester Dashboard Desktop

Scoped strictly to the authenticated user's tickets, displaying Open Tickets, Tickets Waiting for You, Recently Updated, and Recently Resolved:

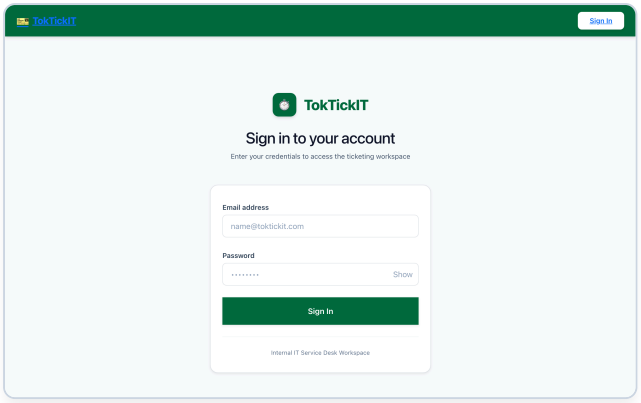


8.2 Requester Dashboard Responsive (Tablet & Mobile)

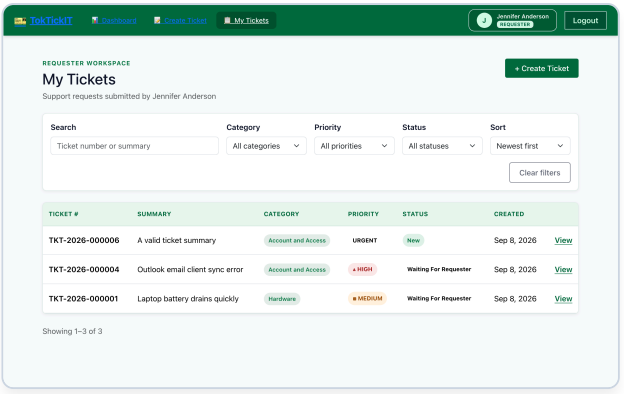
Tablet (768px Viewport)	Mobile (375px Viewport)
	

8.3 Final Regression Evidence (Labs 1–3 Continuity)

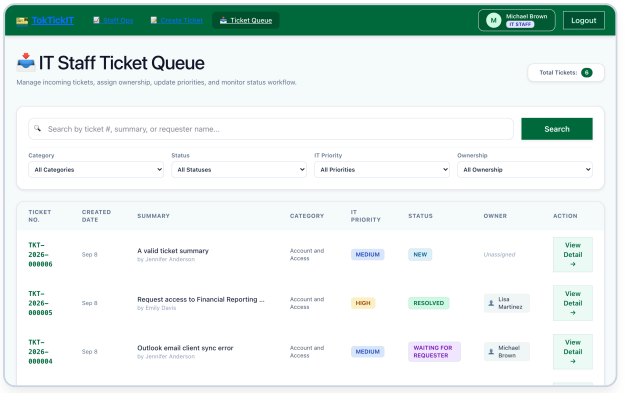
Login & Authentication



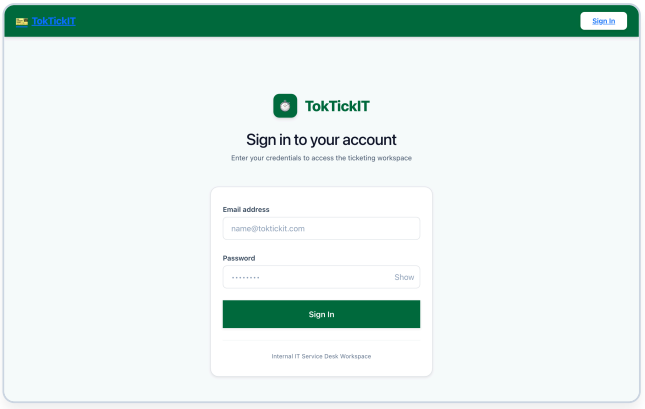
Requester My Tickets



IT Staff Ticket Queue



Admin User Management



Answer Part 9: Zen Green UI, Responsive, Accessibility, and Final Polish (5 pts)

9.1 Design System & Color Palette Compliance

The application strictly adheres to the Zen Green design system:

- **Primary Brand:** `#006B3C` (Zen Forest Green)
- **Deep Accent:** `#064e3b` / `#00542f`
- **Surface & Tint:** `#f0fdf4` / `#e8f5e9`
- **Neutral Dark:** `#0f172a` / `#1e293b`
- **Status Indicators:** Clear non-color cues (icons + text) paired with accessible badge styling.

9.2 Accessibility & Quality Checklist

Requirement	Implementation & Evidence	Verification
Color Contrast	Minimum 4.5:1 contrast ratio across all text elements (WCAG 2.1 AA compliant).	Passed ✅
Visible Focus States	`focus:ring-2 focus:ring-[#006B3C]` on all interactive inputs, buttons, and links.	Passed ✅
Keyboard Operability	All modals, dropdowns, forms, and tables are fully navigable via `Tab`, `Enter`, and `Space`.	Passed ✅
No Horizontal Scroll	Validated at 375px mobile viewport: `scrollWidth === clientWidth` in Playwright E2E.	Passed ✅
No Text Clipping	Responsive CSS grids and flexbox wrapping prevent overlapping controls or truncated text.	Passed ✅
Semantic HTML	Proper `<header>`, `<main>`, `<section>`, `<h1>`–`<h3>`, and ARIA landmark roles used throughout.	Passed ✅
Safe Failure States	Graceful error banners, retry buttons, and empty-state placeholders for zero-data conditions.	Passed ✅
Console Cleanliness	Zero unhandled runtime exceptions, zero syntax warnings, zero dead links.	Passed ✅

Report generated and validated for TokTickIT Sprint 4 Final Submission.